📖 دایرةالمعارف مرجع و صفر تا صد جاوا اسکریپت

دانشنامه کامل و جامع جاوا اسکریپت مدرن

تمام مباحث هسته‌ای، شبکه، ارتباطات سرور، شی‌گرایی، وب استوریج، DOM، بهینه‌سازی و معماری داخلی
با زبان بسیار ساده و کدهای واقعی و تمیز.

۰۱ متغیرها، انواع داده و تفاوت در حافظه (Primitive vs Reference)

تمثیل ساده: متغیرهای اولیه (Primitive) مثل پول نقدی است که در جیبتان دارید و اگر به دوستتان بدهید او پول خودش را دارد. متغیرهای ارجاعی (Object و Array) مثل کلید یک گاوصندوق مشترک هستند؛ هر دو نفر به یک محتوا دسترسی دارند و تغییر یکی روی دیگری هم اثر می‌گذارد!

- **انواع داده اولیه (Primitives):** شامل `string`, `number`, `boolean`, `null`, `undefined`, `symbol`, `bigint` که در حافظه Stack ذخیره می‌شوند.
- **انواع داده ارجاعی (Reference Types):** شامل `Object`, `Array`, `Function` که در حافظه Heap قرار دارند و متغیر فقط آدرس آن را نگه می‌دارد.
- **روش‌های تعریف:** `const` برای مقادیر ثابت (انتخاب اول)، `let` برای متغیرهای قابل تغییر و `var` (روش منسوخ و دارای نشت اسکوپ).

۰۲ متدهای پیشرفته آرایه‌ها (Array Mastery)

تمثیل ساده: کارخانه پردازش کالا! `map` روی تک تک کالاها برچسب قیمت می‌زند، `filter` کالاهای خراب را جدا می‌کند، `reduce` سود کل روز را محاسبه می‌کند، و `some/every` چک می‌کنند که آیا جنسی تاریخ‌گذشته هست یا نه.

```
const cart = [
  { title: "موس", price: 300, count: 2 },
  { title: "کیبورد", price: 700, count: 1 }
];

// جمع کل مبلغ با reduce:
const total = cart.reduce((sum, item) => sum + (item.price * item.count), 0); // 1300

// متدهای غیرمخرب مدرن (ES2023):
const numbers = [3, 1, 4];
const sorted = numbers.toSorted(); // آرایه اصلی دست‌نخورده می‌ماند!
```

۰۳ کالبدشکافی کامل آبجکت‌ها و متدهای مهندسی Object

تمثیل ساده: آبجکت مثل یک کمد با کشوهای دارای اسم است. با متدهای زیر می‌توانید کشوها را قفل کنید، از روی آنها لیست بردارید، یا محتویات را کپی کنید.

استخراج اطلاعات آبجکت

- **Object.keys(obj):** آرایه‌ای از کلیدها
- **Object.values(obj):** آرایه‌ای از مقادیر
- **Object.entries(obj):** جفت کلید و مقدار برای لوپ زدن

امنیت و قفل کردن آبجکت

- **Object.freeze(obj):** فریز کامل (هیچ فیلدی اضافه، حذف یا ویرایش نمی‌شود)
- **Object.seal(obj):** فیلد جدید اضافه نمی‌شود ولی مقادیر قبلی ویرایش می‌شوند

```
const user = { name: "عرفان", role: "توسعه‌دهنده" };
// تبدیل جفت‌های کلید و مقدار به آرایه:
for (const [key, value] of Object.entries(user)) {
  console.log(`${key}: ${value}`);
}
Object.freeze(user); // دیگر قابل تغییر نیست!
```

۰۴ آبجکت Fetch و ارتباط کامل با سرور (HTTP Requests)

تمثیل ساده: متد `fetch()` مثل یک پیک موتوری است! شما آدرس مقصد (URL) و بسته درخواستی‌تان (Headers, Body, Method) را به او می‌دهید، او گاز می‌دهد و می‌رود تا از رستوران پاسخ (Response) را بیاورد.

با fetch می‌توانید تمام متدهای HTTP مثل `GET` (دریافت)، `POST` (ارسال داده)، `PUT` (ویرایش کامل)، `PATCH` (ویرایش جزئی) و `DELETE` (حذف) را اجرا کنید.

```
// ۱. ارسال اطلاعات (POST) همراه هدر و بدنه JSON:
async function createNewUser() {
  try {
    const response = await fetch("https://api.example.com/users", {
      method: "POST",
      headers: {
        "Content-Type": "application/json",
        "Authorization": "Bearer my-secret-token"
      },
      body: JSON.stringify({ name: "عرفان", role: "معمار نرم‌افزار" })
    });

    // بررسی وضعیت پاسخ:
    if (!response.ok) throw new Error(`خطای سرور: ${response.status}`);
    const result = await response.json();
    console.log("کاربر ساخته شد:", result);
  } catch (err) {
    console.error("خطا در شبکه یا اتصال:", err.message);
  }
}
```

💡 **لغو درخواست با AbortController:** اگر کاربر صفحه را بست یا سرچ جدیدی زد، می‌توانید درخواست قبلی را لغو کنید تا بیهوده اینترنت مصرف نشود:

```
const controller = new AbortController(); fetch(url, { signal:
controller.signal }); controller.abort();
```

۰۵ کلاس‌ها (Classes) و الگوهای شی‌گرایی (OOP)

تمثیل ساده: کلاس مثل **نقشه ساخت کارخانه** است. با یک نقشه می‌توانید ۱۰۰۰ خودرو (Instance) بسازید که همگی ترمز و فرمان دارند، اما رنگ و پلاکشان متفاوت است.

```
class Car {
  #chassisNumber; // متغیر کاملاً خصوصی (Private Field) با علامت هشتگ

  constructor(brand, color, chassis) {
    this.brand = brand;
    this.color = color;
    this.#chassisNumber = chassis;
  }

  drive() {
    return `${this.brand} شروع به حرکت کرد!`;
  }
}

// ارث‌بری (Inheritance):
class ElectricCar extends Car {
  constructor(brand, color, chassis, battery) {
    super(brand, color, chassis);
    this.battery = battery;
  }
}
```

۰۶ ذخیره‌سازی در مرورگر (LocalStorage, SessionStorage, Cookies, IndexedDB)

LocalStorage (حافظه ابدی)

حداکثر ۵ تا ۱۰ مگابایت. حتی اگر مرورگر یا ویندوز خاموش و روشن شود، داده‌ها پاک نمی‌شوند. (عالی برای تم تاریک/روشن و تنظیمات کاربر).

SessionStorage (حافظه تب فعال)

فقط تا زمانی که کاربر آن تب مرورگر را باز نگه داشته زنده است. با بستن تب، اطلاعات پاک می‌شود. (عالی برای اطلاعات یک فرم موقت).

کوکی‌ها (Cookies)

حجم کم (۴ کیلوبایت)، دارای تاریخ انقضا و با هر بار درخواست به سرور همراه هدر فرستاده می‌شوند. (عالی برای توکن‌های لاگین امن HttpOnly).

دیتابیس کامل IndexedDB

یک پایگاه‌داده NoSQL قدرتمند درون مرورگر برای ذخیره صدها مگابایت داده آفلاین، تصاویر و کش پیشرفته وب اپلیکیشن‌ها (PWA).

```
// ذخیره شیء در لوکال استوریج با فرمت متنی JSON:
localStorage.setItem("theme", "dark");
localStorage.setItem("profile", JSON.stringify({ name: "عرفان" }));

// خواندن و تبدیل مجدد به آبجکت جاوا اسکریپت:
const profile = JSON.parse(localStorage.getItem("profile"));
```

۰۷ رویدادها و انتشار آن‌ها (Event Bubbling & Capturing)

تمثیل ساده: مثل حباب هوای زیر آب است! وقتی روی یک دکمه داخل یک کارت کلیک می‌کنید، رویداد کلیک ابتدا روی دکمه منفجر می‌شود، سپس به سمت بالا بالا می‌آید و کارت، بخش و در نهایت کل صفحه را خبردار می‌کند (Event Bubbling).

- **متوقف کردن حباب (`e.stopPropagation()`):** مانع بالا رفتن رویداد به والدین می‌شود.
- **جلوگیری از رفتار پیش‌فرض (`e.preventDefault()`):** مانع ریفرش شدن صفحه هنگام سابمیت فرم یا پرش لینک می‌شود.
- **الگوی Event Delegation:** به جای وصل کردن ۱۰۰۰ لیسنر به ۱۰۰۰ سطر جدول، فقط یک لیسنر به والد (جدول) وصل می‌کنید و با `e.target` می‌فهمید کدام سطر کلیک شده است! فوق‌العاده سبک برای رم.

۰۸ کلوژر، اسکوپ، Hoisting، و Event Loop (مرور یکپارچه عمیق)

کلوژر (Closure)

توانایی تابع برای به خاطر سپردن محیط تولدش و متغیرهای خصوصی حتی پس از اتمام اجرای تابع مادر.

حلقه رویداد (Event Loop)

اولویت قطعی پردازش: ابتدا کدهای همگام در Call Stack، سپس اورژانسی‌ها در Microtask Queue (پرامیس‌ها)، و در آخر صف Callback Queue (تایمرها).

۰۹ آبجکت Proxy و Reflect (متاپروگرامینگ و زیربنای ری‌اکتیویتی)

تمثیل ساده: `Proxy` مثل یک بادیگارد جلوی آبجکت شماست! هر کس بخواهد به اطلاعات آبجکت نگاه کند یا مقداری را تغییر دهد، اول باید از گیت بازرسی بادیگارد رد شود. (این همان جادویی است که فریم‌ورک‌های مدرن برای باخبر شدن از تغییر متغیرها استفاده می‌کنند!)

```
const target = { price: 100 };
const proxy = new Proxy(target, {
  set(obj, prop, value) {
    if (prop === 'price' && value < 0) {
      throw new Error("قیمت نمی‌تواند منفی باشد!");
    }
    obj[prop] = value;
    return true;
  }
});
```

۱۰ محاسبات سنگین در پس‌زمینه با Web Workers

چون جاوا اسکریپت تک‌نخه است، اگر پردازش ریاضی ۱۰ ثانیه‌ای انجام دهید، کل مرورگر کاربر قفل می‌شود. **Web Workers** اجازه می‌دهد پردازش را به یک هسته دیگر CPU در پس‌زمینه بفرستید تا صفحه وب کاملاً روان و بدون لگ باقی بماند.